

操作系统

第二章 进程管理

第二部分

进程状态、状态转换 与 调度 (1)

同济大学计算机系



Part 1、多道系统中的进程状态

- 为什么多道？
- 进程调度状态
- 进程状态转换图

进程的执行

- 输入阶段、计算阶段、输出阶段 交错
- 输入阶段，使用输入设备，将新数据读入进程图像（数据段、堆栈段）
 - 读磁盘文件、网络链接（socket）、接收其它进程传来的数据
- 计算阶段，使用CPU，处理读入新数据，生成运行结果
- 输出阶段，使用输出设备，将运行结果存盘 或者 送给其它进程处理

输入输出设备

- 输入设备
 - 磁盘、网络、终端（键盘）
- 计算阶段，处理读入新数据，生成运行结果
- 输出阶段，将运行结果存盘 或者 送给其它进程处理
 - 磁盘、网络、终端（屏幕）

进程 与 外设的同步



处理数据



数据提供者 & 传输通道

事实:

- 1、输入操作没完成，没有可供处理的新数据，进程没法运行
- 2、输出操作没完成，生成的新数据没地方存放，进程也没办法运行

输入、输出操作很慢，比CPU访问内存慢若干数量级。所以，绝大多数情况下，是运算(CPU) 等 输入输出(外设)。



0、为什么多道?

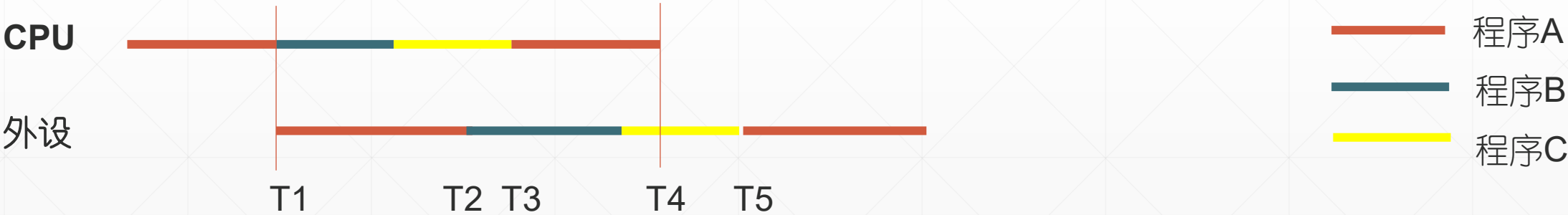


单道系统和多道系统的运行情况

单道系统一次只能跑1个程序，这个程序运行完毕，系统才能装入、运行其它程序。



多道系统可以同时跑很多程序。正在运行的程序IO时，让出CPU给其它进程运算。



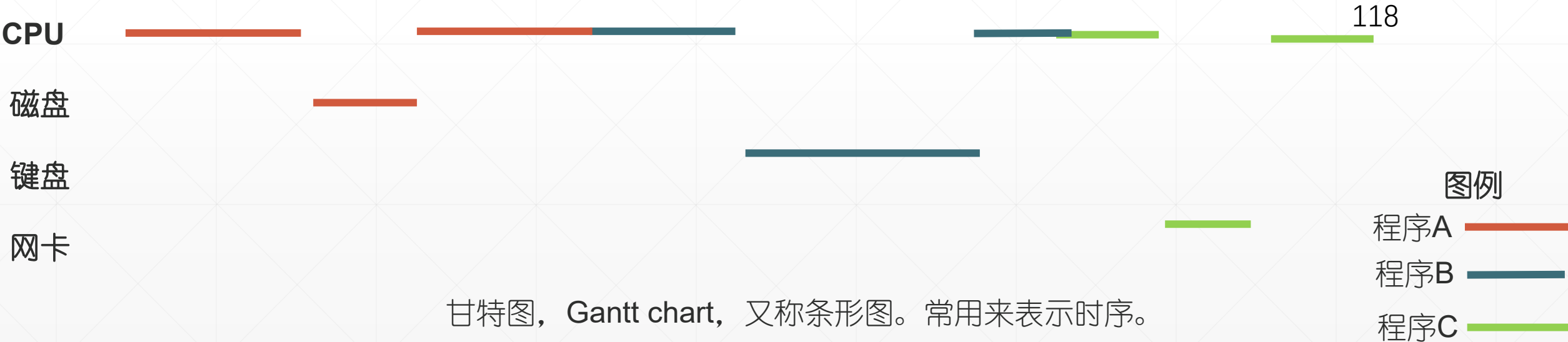
[T1, T4] 外设和CPU全忙，跑完这3个程序，总耗时减少 $T4 - T1$

使用同一个外设



例1：某单道系统， 需要执行3个程序。先来先服务。
画时序图。计算完成所有计算任务的总耗时（118ms）。

程序	进入系统的时刻	第一轮计算时间	外设IO时间 (使用不同的外设)	第二轮计算时间 (之后结束)
A	0	20	10（读磁盘）	20
B	10	15	20（读键盘）	10
C	20	5	8（读网卡）	10



甘特图， Gantt chart， 又称条形图。常用来表示时序。

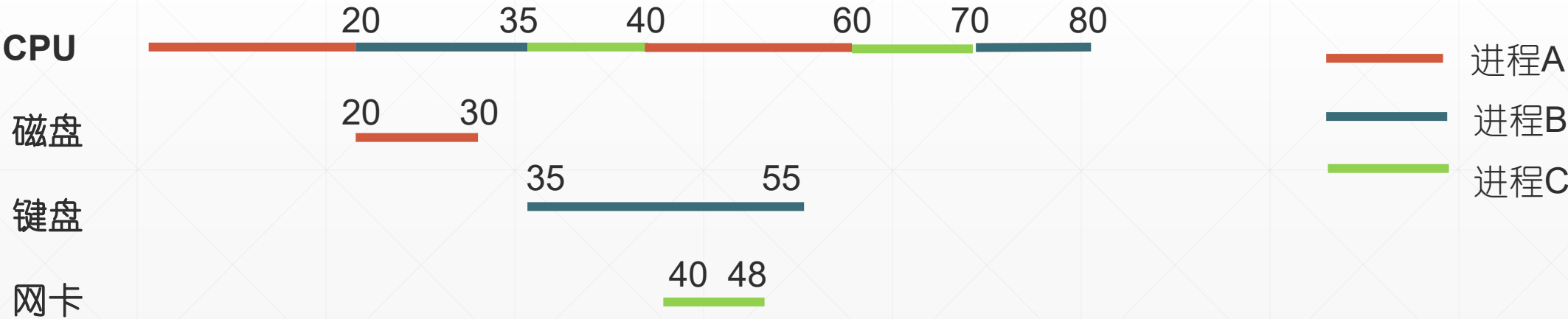


规则： 启动IO，程序放弃CPU。
IO完成后，程序可以使用CPU继续运行

例2：多道程序同时运行（先来先服务）

■ 80ms

程序	进入系统的时刻	第一轮计算时间	外设IO时间	第二轮计算时间（之后结束）
A	0	20	10（读磁盘）	20
B	10	15	20（读键盘）	10
C	20	5	8（读网卡）	10



总结

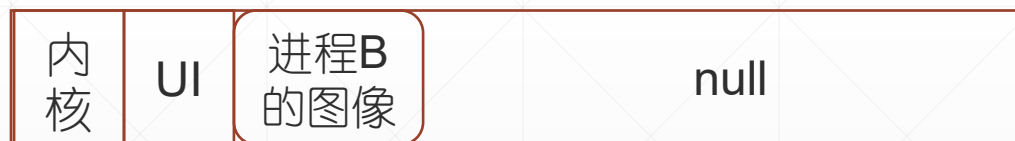
- 单道系统的缺点：↵
 - 无论何时，至多一块芯片在工作。IO、计算不能并行。↵
 - 系统资源利用率低。吞吐率低。↵
 - 单道系统的优点：↵
 - 程序独占计算机系统。从开始运行至运行结束，耗时最短。
↵
 - 多道系统的优点：↵
 - 系统（所有资源）利用率高，吞吐率高。↵
- 优点，来自计算、IO 并行的设计。↵

最后。内存中存放好多程序供 CPU 选择是有好处的。CPU 空闲时，找到没在 IO 可供执行的程序，概率高。有利于保持 CPU 忙碌。另一方面，忙碌的 CPU 会执行更多的应用程序，发出更多的 IO 命令，从而保持 IO 设备忙碌。↵

内存装的程序越多，越有可能让所有设备保持忙碌，从而提高系统的吞吐量。这里有一个度，需要对系统中并发的进程总数进行控制。↵

多道系统的内存中并发很多进程图像

单道



多道



管理并发进程，多道系统必需具备以下功能：

1. 能够运行可重定位的程序（编译好的程序，不出现物理地址），确保无论把程序放在哪里，它都能跑起来
2. 为程序分配内存 **空间分配**
3. 保证运行中的进程不会破坏操作系统，不会访问分配给其它进程的内存空间 **隔离**
4. 安排多个程序的执行的先后次序 **调度**
5. 设置CPU，执行新选中进程 **进程切换（完成一半，为新选中的进程建立地址映射关系）**
6. 保证任何用户未经授权不得访问其它用户的软件资源（程序和数据文件） **安全（权限检测和 身份认证）**
7. 实现应用系统的控制逻辑，比如 sleep(10), fork, pthread **进程控制**



用来提升系统性能，改善用户体验的其它功能：中断、DMA、缓存

典型的多道操作系统：Windows, Unix, Linux

1、任意时刻，一颗CPU，只能运行一个进程

- MMU引用的页表，反映该进程虚空间布局 & 地址映射关系
- 执行单元正在执行该进程的一个子程序。
 - 通用寄存器是这个子程序的执行现场：
 - EIP，下条指令
 - ESP、EBP定位这个子程序的栈帧
 - EAX，存放这个子程序的返回值
 - EFLAGS中的bit，标识这个程序的运行情况
 - ZF为1，算数运算结果为0
 - IF为1，当前正在执行的代码，开中断运行。
0，当前正在执行的代码，关中断运行，CPU不响应中断。
 -
 - CS，最后2位是该进程的执行状态
 - 00，核心态运行。正在执行系统调用。。。
 - 11，用户态运行。正在执行应用程序

2、进程调度状态

- CPU正在执行的进程，运行态。
- CPU不能执行的进程，阻塞态。
- 等待执行的进程，就绪态。

每个阻塞进程，有确切的阻塞原因：

- 没有可供处理的新数据
 - 磁盘IO操作没完成
 - 等待键盘输入数据
 - 等待网络包

} & 数据缓存块
- 有数据，但不能处理
 - 数据被别的进程锁住了

& 锁变量
- 定时器没到期
 - sleep(10)，之后的代码要10s之后才能跑

& 定时器变量

每个阻塞进程，阻塞原因表示为：

- 0，没有阻塞
- 非0，是一个内核变量的首地址

案例：Unix V6++进程的调度状态

```
class Process
{
public:
    enum ProcessState
    {
        SNULL      = 0,    // 空元素
        SSLEEP      = 1,    // 阻塞，高优先权睡眠
        SWAIT        = 2,    // 阻塞，低优先权睡眠
        SRUN         = 3,    // 运行 或 就绪
        SIDL         = 4,    // 进程创建中，不接受调度
        SZOMB        = 5,    // 进程已终止，仅剩PCB
        SSTOP        = 6,    // 被debug的进程
    };

public:
    .....
    ProcessState p_stat;    // 进程调度状态
    unsigned long p_wchan;  // 睡眠原因
    .....
}
```

读写磁盘数据，阻塞

其它原因，阻塞

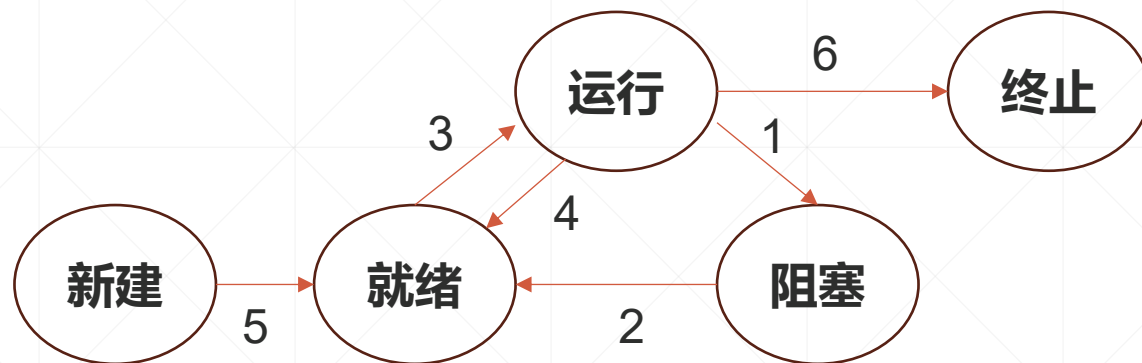


例 2

单核计算机系统，并发20个进程。
运行态进程最多几个？最少几个？
阻塞态进程，最多几个？最少几个？
就绪态进程最多几个？最少几个？

4核计算机系统，并发20个进程。
运行态进程最多几个？最少几个？
阻塞态进程，最多几个？最少几个？
就绪态进程最多几个？最少几个？

3、进程状态变迁



- 1、入睡（阻塞）：进程无法前进主动放弃CPU
- 2、唤醒：IO完成.....进程恢复可执行状态。
- 3、被调度：被选中成为现运行进程。
- 4、被抢占：系统中出现优先级更高的进程。
- 5、进程图像创建完毕，就绪态
- 6、应用程序执行完毕或被杀死

入睡原因

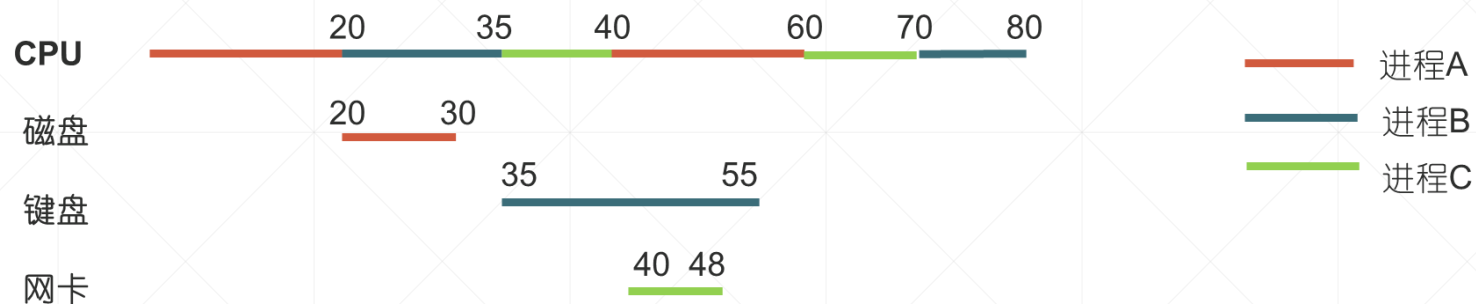
- 1) 启动IO操作
- 2) sleep(10), 启动定时器
- 3) 内存中没有新数据
- 4) 进程访问的数据被其它进程锁住了
- 5) 排队等待资源

唤醒事件

- 1) IO完成
- 2) 定时器到期, 10s后进程就绪
- 3) 新数据就绪 (前驱任务完成 或 IO操作完成)
- 4) 对方数据访问完毕, 解锁
- 5) 获得使用权

例3:

程序	进入系统的时刻	第一轮计算时间	外设IO时间	第二轮计算时间（之后结束）
A	0	20	10（读磁盘）	20
B	10	15	20（读键盘）	10
C	20	5	8（读网卡）	10



	18s	23s	32s	36s	45s	65s
进程A	运行	阻塞	就绪	就绪	运行	-（已终止）
进程B	就绪	运行	运行	阻塞	阻塞	就绪
进程C	-	就绪	就绪	运行	阻塞	运行



例4：进程的状态？

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY);    // 1
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
    while( read(oldFile,&c,1) == 1 )    // 2
        write(newFile,&c,1);    // 3

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

例5:

- 阅读、编写方阵自乘程序，回答以下问题

```
# include <stdlib.h>
# include <stdio.h>

# define M 16
int version = 1;
int matrixOriginal[M][M];
int matrixDes [M][M];
```

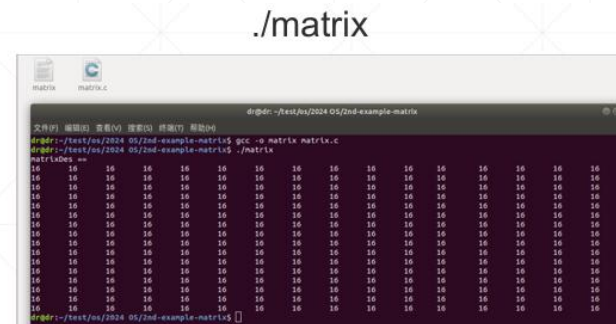
```
void produce ( int row, int column )
{
    int i;
    for (i=0;i<M;i++)
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
}
```

```
int main()
{
    int i , j;
```

```
    for (i=0;i<M;i++)
        for (j=0;j<M;j++) {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
        }
```

```
        for (i=0;i<M;i++)
            for (j=0;j<M;j++)
                produce(i , j);
```

```
        printf("matrixDes == \n");
        for (i=0;i<M;i++) {
            for (j=0;j<M;j++)
                printf("%d\t", matrixDes[i][j]);
            printf("\n");
        }
    }
```



- 1、matrix程序的代码，包括？
- 2、指出matrix程序的只读数据，全局变量和局部变量。
- 3、M是什么（是编译器使用的符号，不是应用程序的数据）？请将M修改为4096，测速。为什么程序慢那么多？加速它。



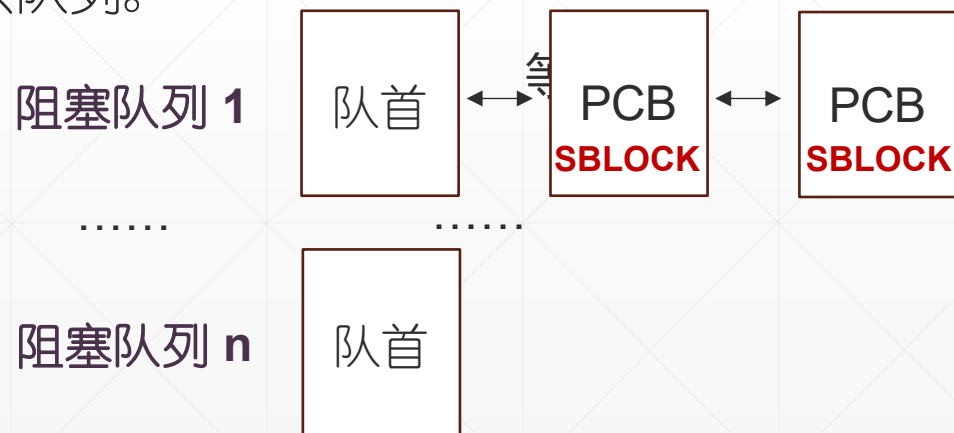
4、进程队列（现代操作系统具备，Unix V6++暂无）

单核系统中的进程队列

- 管理可执行进程（就绪、现运行），用就绪进程队列。
 - 系统只有一根就绪进程队列。现运行进程放弃CPU时，从就绪队列中挑选新运行进程。

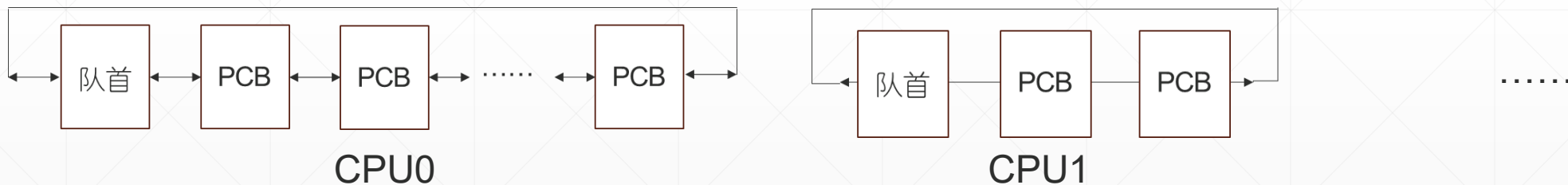


- 管理阻塞进程，用阻塞进程队列。一个阻塞原因，一条队列。
 - 磁盘的IO请求队列，排着等待执行IO操作，或者待IO操作完成的进程。
 - 信号量队列，排着等待访问共享资源的进程。
 - 时钟队列，排着等待定时器到期的进程。
 -有多少种入睡理由，就有多少根阻塞队列。



多核系统中的进程队列

- 可以只配一根就绪进程队列，比如，早期Linux（指Linux2.6之前）。
- 优化的实现，为每颗CPU配一个就绪队列。
 - 现运行进程放弃CPU后，从自己的就绪队列挑优先级最高的进程
 - 如果就绪队列长度差值过大，执行负载均衡操作，将一些进程移到长度短的队列



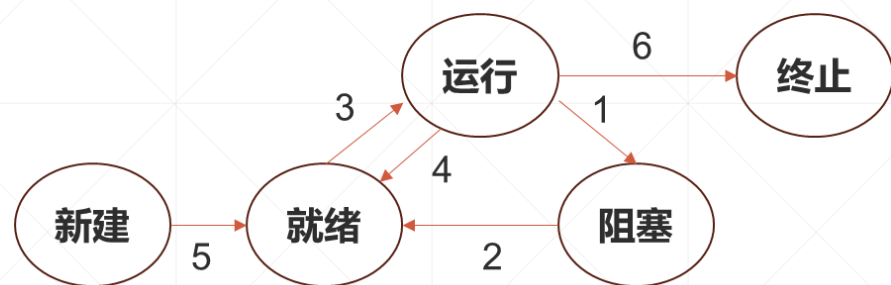
- 管理阻塞进程，用阻塞进程队列。
 - 同，单核系统



0#进程

- 每颗CPU配一个0#进程，没有就绪进程可供执行时，运行0#进程，CPU空转 (idle)

5、进程状态转换，阻塞



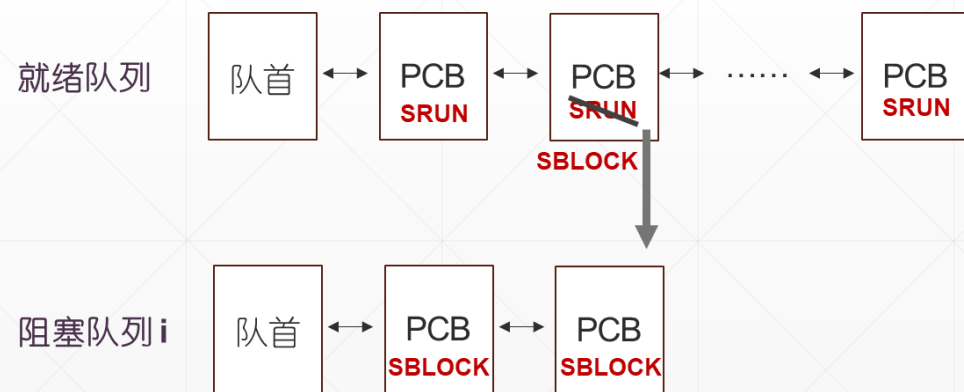
1、修改调度状态

运行→阻塞

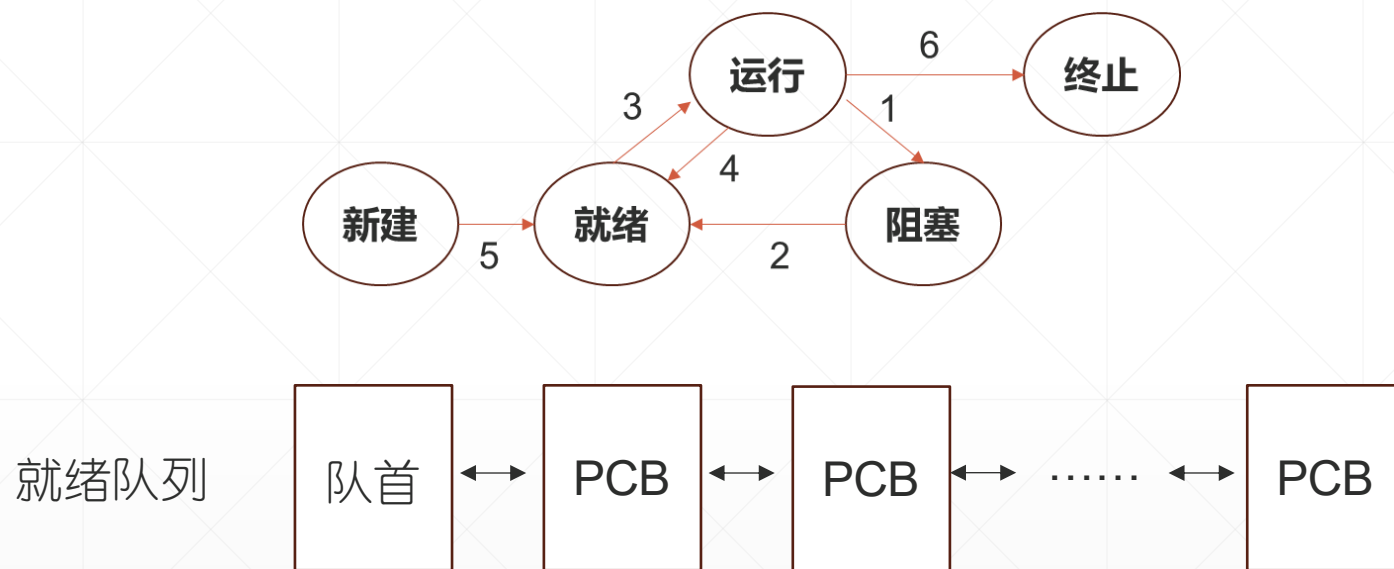
2、对PCB执行队列操作

从就绪队列移除，插入阻塞队列

3、保护现场（存入PCB或核心栈），放弃CPU

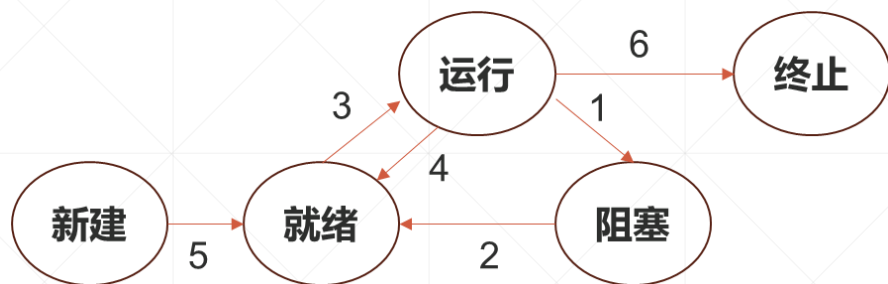


调度



遍历就绪队列，找优先级最高进程
恢复现场，建立地址映射关系

唤醒

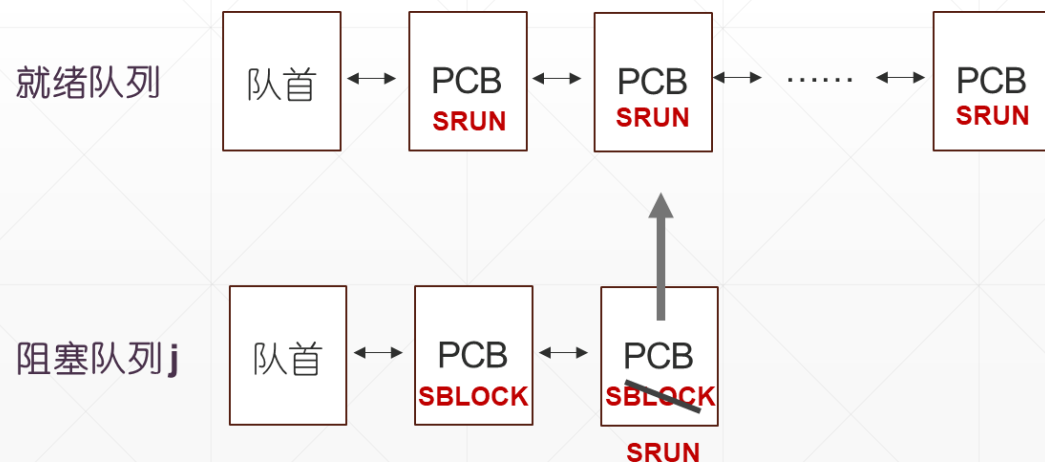


1、修改目标进程调度状态

阻塞→就绪

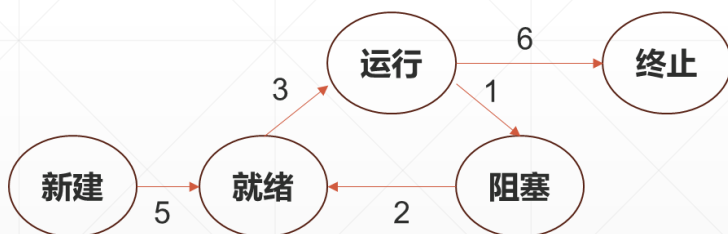
2、对目标进程PCB执行队列操作

从阻塞队列移除，插入就绪队列，等待调度

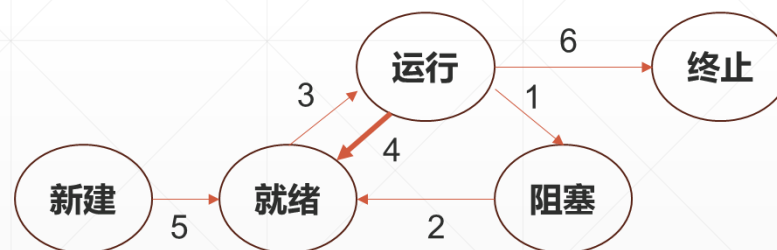


5、调度点

- 多道系统，进程轮流使用CPU。现运行进程必须在恰当的时刻，将CPU让与他人。这些时刻，是系统的调度点。
- 不同的调度策略，不同的调度点
 - 不可剥夺的系统 (Non Preemptable)
 - 可剥夺的系统 (Preemptable)

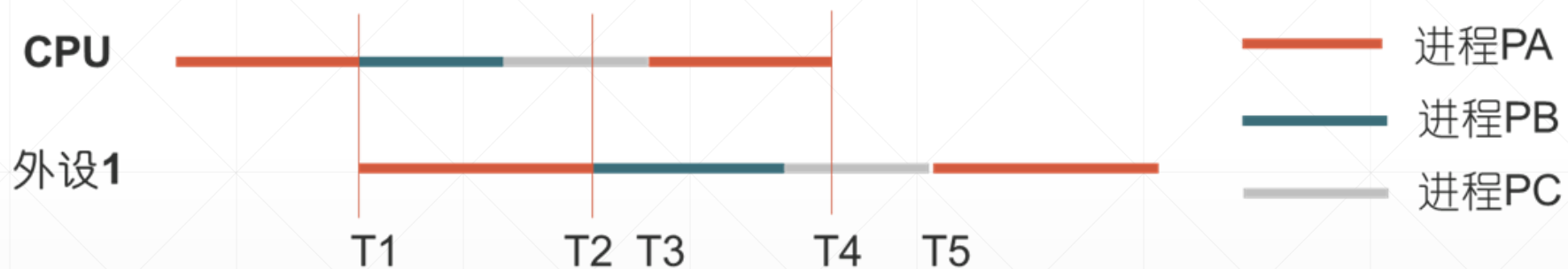


现运行进程，如若不是终止或入睡，不放弃CPU



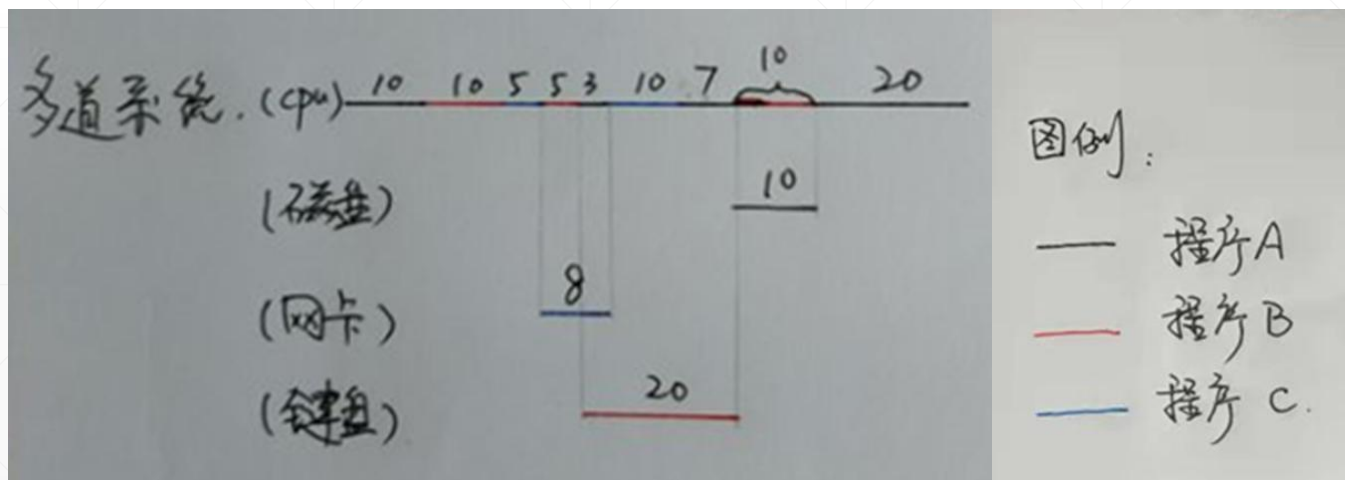
出现优先级更高的进程
现运行进程放弃CPU，让位

Q: 可剥夺还是不可剥夺？



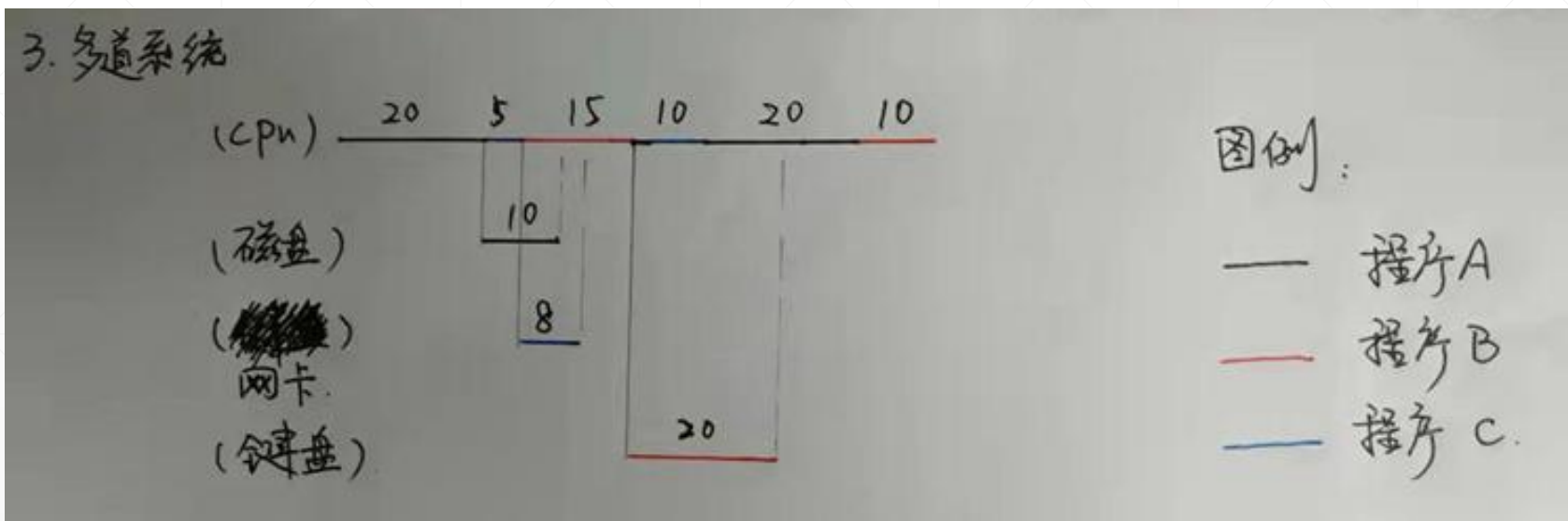
例 6、 多道方式运行。采用优先级高者先执行的调度策略。优先级： 进程A<进程B<进程C。如果出现优先级更高的进程，正在执行的进程让出CPU。

程序	进入系统的时间	第一轮计算时间	外设IO时间	第二轮计算时间（之后结束）
A	0	20	10（读磁盘）	20
B	10	15	20（读键盘）	10
C	20	5	8（读网卡）	10

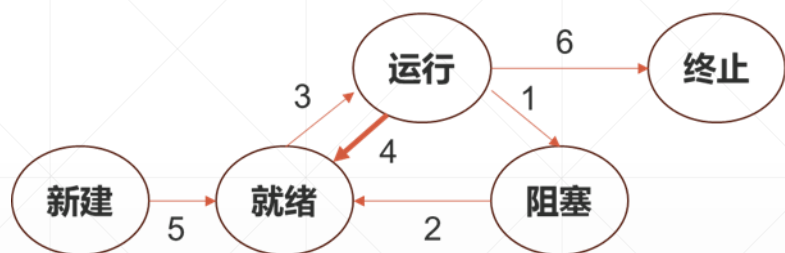


例 7、多道方式运行。采用优先级高者先执行的调度策略。优先级： 进程A<进程B<进程C。
出现优先级更高的进程，正在执行的进程不会让出CPU，会一直运行直至IO或运行结束。

程序	进入系统的时间	第一轮计算时间	外设IO时间	第二轮计算时间（之后结束）
A	0	20	10（读磁盘）	20
B	10	15	20（读键盘）	10
C	20	5	8（读网卡）	10



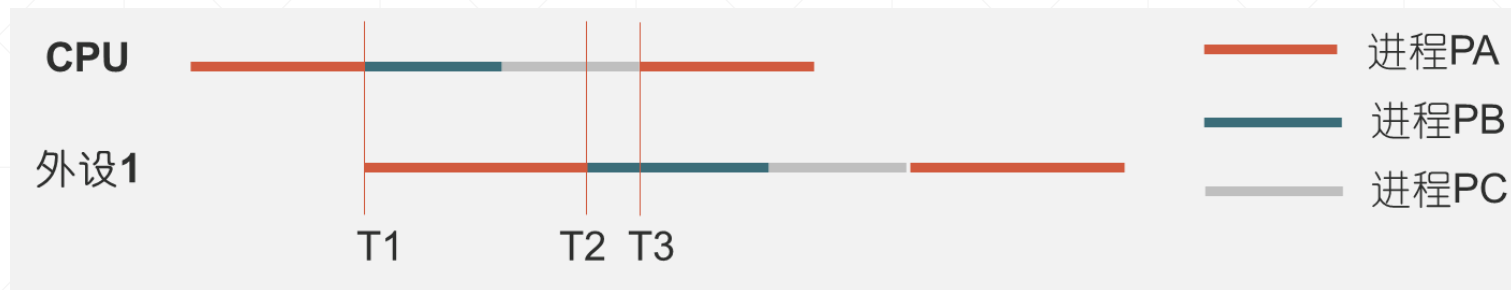
Unix V6++采用可剥夺的调度策略



- 现运行进程主动放弃CPU
 - 入睡
 - 终止
 - 从核心态返回用户态时，现运行进程有可能被剥夺
 - 有优先级更高的就绪进程，进程切换，现运行进程让出CPU
 - 没有，现运行进程返回用户态执行应用程序
- 涉及中断响应和返回，我们慢慢来

Unix V6++的进程状态转换 1、入睡（状态变迁1）

- T1时刻，现运行进程PA执行Sleep()函数入睡



```
void Process::Sleep(unsigned long chan, int pri)
```

```
{
```

```
.....
```

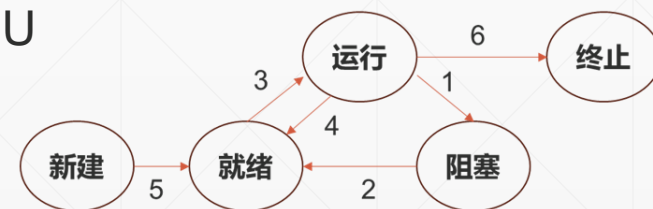
```
this->p_wchan = chan; // 1、设置睡眠原因。通常，chan是一根指针，指向睡眠事件关联的内核变量
```

```
this->p_stat = Process::SSLEEP; // 2、改变进程的调度状态
```

```
.....
```

```
Kernel::Instance().GetProcessManager().Swch(); // 3、进程PA放弃CPU
```

```
}
```

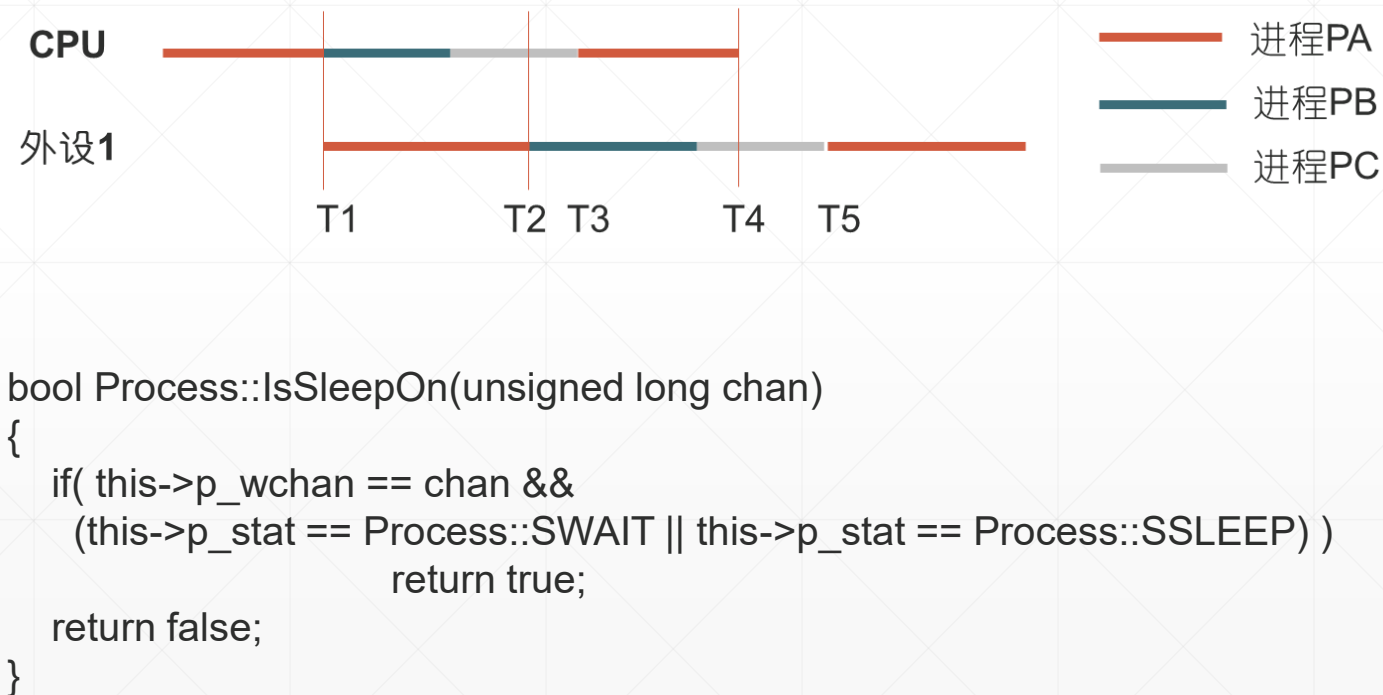


Unix V6++的进程状态转换

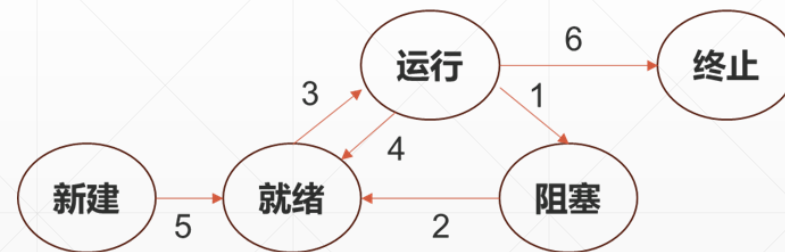
2、唤醒

- T2时刻，PA进程完成IO操作。
现运行进程PC执行WakeUpAll()函数，
唤醒睡眠进程PA。

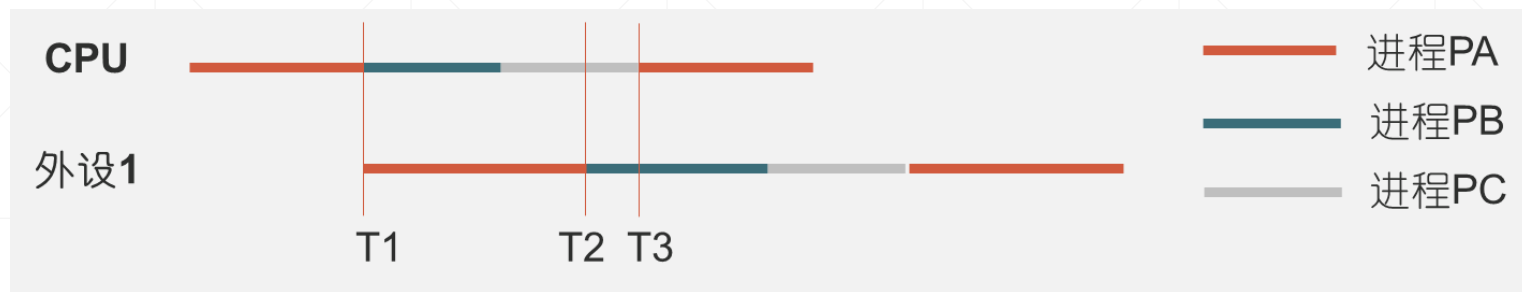
```
void ProcessManager::WakeUpAll(unsigned long chan)
{
    // 唤醒p_wchan==chan的所有进程
    for(int i = 0; i < ProcessManager::NPROC; i++)
        if( this->process[i].IsSleepOn(chan) )
            this->process[i].SetRun();
}
```



```
void Process::SetRun()  
{  
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();  
  
    /* 清除睡眠原因，转为就绪状态 */  
    this->p_wchan = 0;  
    this->p_stat = Process::SRUN; // PA变就绪了，可以接受调度  
    .....  
}
```



3、T3时刻的进程切换：现运行进程PC将CPU让给PA



- PC保护现场
- 系统选中PA（系统中有2个SRUN进程，PA优先级高）
- 为PA设置CPU运行环境，让它成为**现运行进程**
 - 恢复现场
 - 为MMU建立PA的地址映射关系。
- PA恢复运行，接上次断点

T3时刻, Swtch()函数的一次执行实例

```
int ProcessManager::Swtch()
{
```

```
    User& u = Kernel::Instance().GetUser();    // 现运行进程PC的user结构
```

// 1、现场保护

```
    SaveU(u.u_rsav);
```

```
    .....
```

```
}
```

宏展开后, 是两条指令

```
movl %esp, u.u_rsav[0]
```

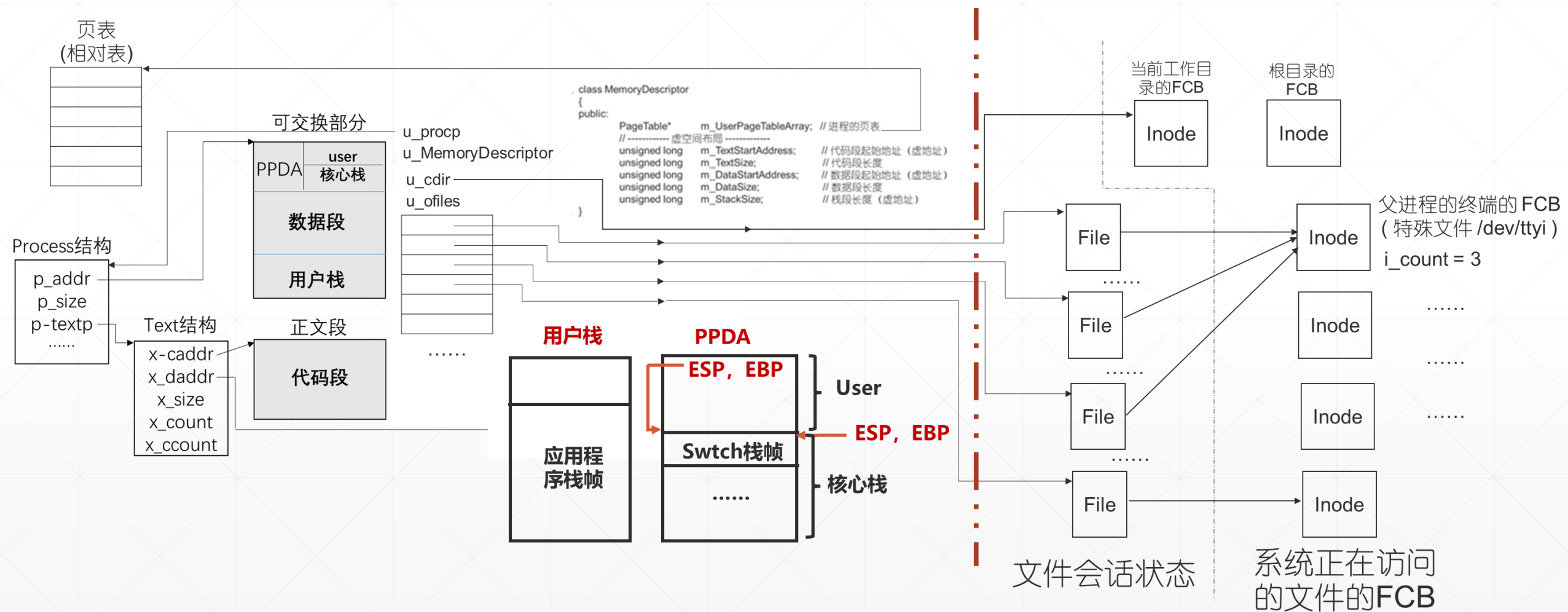
```
movl %ebp, u.u_rsav[1]
```

将寄存器EBP和ESP存入PC进程的user结构 (u.u_rsav数组)

```
#define SaveU(u_sav) \
    __asm__ __volatile__( \
        "movl %%esp, %0\n\t" \
        "movl %%ebp, %1\n\t" \
        : "+m"(u_sav[0]), "+m"(u_sav[1]) \
        );
```

```
class User
{
    .....
public:
    unsigned long u_rsav[2];    /* 用于保存esp与ebp指针 */
    .....
}
```

Swch()函数为进程PC制作的可执行进程图像



T3时刻, Swtch()函数的一次执行实例

```
int ProcessManager::Swtch()  
{
```

```
.....
```

// 2、0#进程执行select()函数, 挑选优先级最高的就绪进程

```
Process* procZero = &process[0];
```

```
X86Assembly::CLI();
```

```
SwtchUStruct(procZero);
```

```
RetU();
```

```
X86Assembly::STI();
```

```
Process* selected = Select();
```

```
.....
```

```
}
```

让0#进程成为现运行进程

```
32 #define SwtchUStruct(p) \
33     Machine::Instance().GetKernelPageTable().m_Entrys[Kernel::USER_PAGE_INDEX].m_PageBaseAddress \
34     = (p)->p_addr / PageManager::PAGE_SIZE; \
35     FlushPageDirectory();
```

系统核心页表[1023].m_PageBaseAddress = process[0].p_addr / 4096;
清除TLB (缓存的所有U=1的PTE)

T3时刻, Swtch()函数的一次执行实例

```
int ProcessManager::Swtch()  
{
```

.....

// 2、0#进程执行select()函数, 挑选优先级最高的就绪进程

```
Process* procZero = &process[0];
```

```
X86Assembly::CLI();
```

```
SwtchUStruct(procZero);
```

```
RetU();
```

```
X86Assembly::STI();
```

```
Process* selected = Select();
```

.....

```
}
```

```
50 #define RetU() \
51     __asm__ __volatile__ ("    movl %0, %%eax;
52                             movl (%%eax), %%esp;
53                             movl 0x4(%%eax), %%ebp;"
54                             :
55                             : "i" (0xc03ff000) );
```

\ 0#进程从user结构中
\ 取出保存的ESP和
\ EBP, 恢复核心栈

宏展开:

```
movl 0xc03ff000, %eax;
```

```
movl (%eax), %esp;    // ESP = u_rsav[0], 恢复栈顶
```

```
movl 4(%eax), %esp;   // EBP = u_rsav[1], 恢复栈顶帧基地址
```

{

static int lastSelect = 0;

while (true)

{

this->RunRun = 0;

int priority = 256;

int best = -1;

for (int count = 0; count < NPROC ; count++)

{

int i = (lastSelect + 1 + count) % NPROC;

if (Process::SRUN == process[i].p_stat && (process[i].p_flag & Process::SLOAD) != 0)

if (process[i].p_pri < priority)

{

best = i;

priority = process[i].p_pri;

}

}

if (-1 == best)

{

__asm__ __volatile__ ("hlt");

continue;

}

this->CurPri = priority;

lastSelect = best;

return &process[best];

}

{

Select()函数选新运行进程



算法将Process数组维护成循环队列。

lastSelect是当前指针，现运行进程的下标。

- 系统初始化时，置0。
- 进程切换时，从下个元素开始，寻找图像在内存中的、优先级最高的就绪进程。

运行在0#进程核心栈。

遍历Process数组，找p_pri值最小的进程，PA被选中

返回 PA Process结构的首地址

T3时刻, Swtch()函数的一次执行实例

```
int ProcessManager::Swtch()
{
```

```
.....
```

```
X86Assembly::CLI();
```

```
SwtchUStruct(selected); // 3.1 为 PA 的 PPDA区 ( user 和 核心栈 ) 建立地址映射关系
```

```
RetU(); // 3.2 从 user 结构中恢复 PA核心栈顶帧
```

```
X86Assembly::STI();
```

```
User& newu = Kernel::Instance().GetUser(); // 取 PA进程的 user结构
```

```
newu.u_MemoryDescriptor.MapToPageTable(); // 3.3 为 PA 的用户空间建立地址映射关系
```

```
.....
```

```
return 1; // Swtch返回, PA进程执行Sleep函数
```

```
}
```

